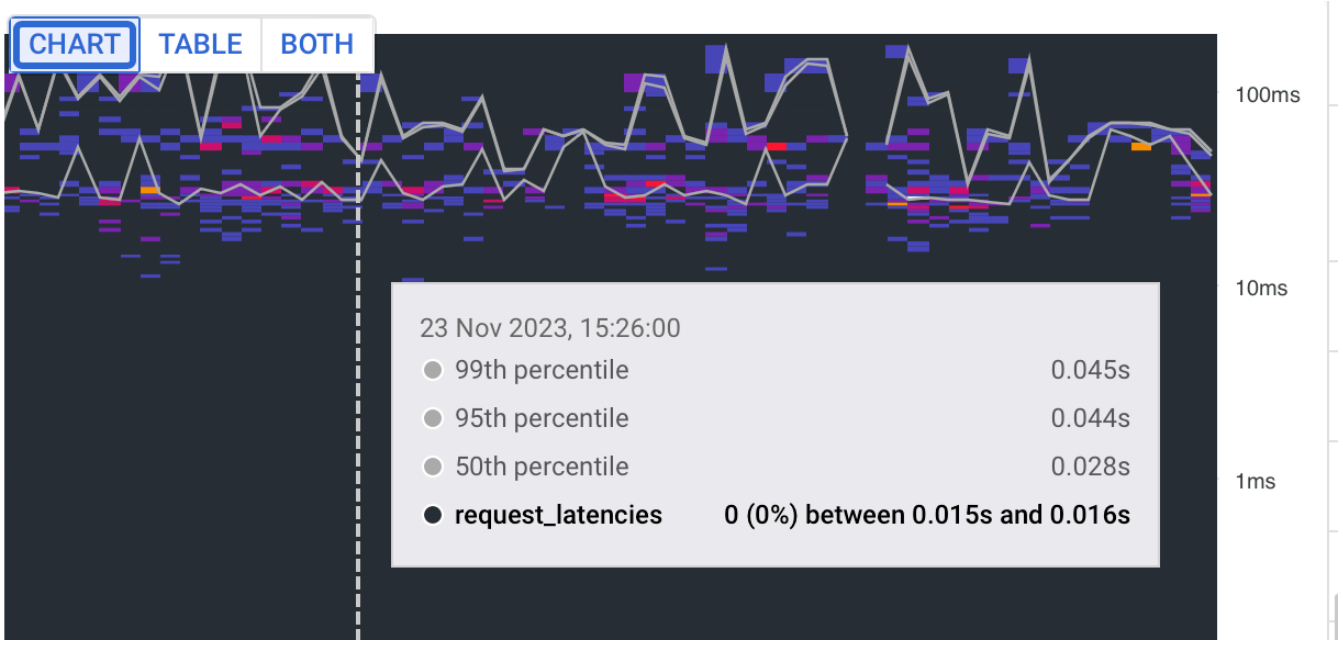
Non-functional Requirement in System Design

systemdesignschool.io/fundamentals/latency



Latency

Latency is an essential non-functional requirement. It plays a crucial role in shaping the user experience and overall system performance. But what is latency, exactly? Simply put, latency refers to the delay that occurs from the time a system receives a request until it responds to that request.

Latency: The Basics

In the simplest terms, latency is a delay. In system design, it is a measure of the time it takes for a bit of data to travel from one place to another in a network. The units for measuring latency are usually milliseconds (ms) or microseconds (µs).

High latency leads to a slow and inefficient system, which can negatively impact user satisfaction, particularly in real-time applications like video streaming, online gaming, or high-frequency trading, where delays of even a few milliseconds can be detrimental.

Understanding Latency and Response Time

A key part of understanding latency is recognizing its relationship to response time. From a client's perspective, the total latency (also called response time) consists of two main components:

- **Network latency**: This refers to the time taken for a request and the subsequent response to travel across the network - from the client's machine to the server, and then back again. This is also known as time spent 'on the wire'.
- **Server-side latency**: This is the time taken by the server to understand the request, process it, and prepare a response.

So, when we talk about total latency or response time, we are essentially adding up the network latency and the server-side latency.

Factors Contributing to Latency

As mentioned above, the total latency observed by the client is the sum of network latency and server-side latency. The key to managing latency, therefore, lies in understanding and optimizing these two factors:

Network Latency: It’s a measure of the time taken for a packet of data to get from one point to another. Network latency could be affected by factors such as the physical distance the data has to travel, the number of nodes it has to pass through, and the speed of the network.

Server-Side Latency: This represents the time taken by a server to process a request. The processing could involve calculations, data retrieval, or various other tasks. Server-side latency could be influenced by factors such as server hardware, software efficiency, and the complexity of the requested task.

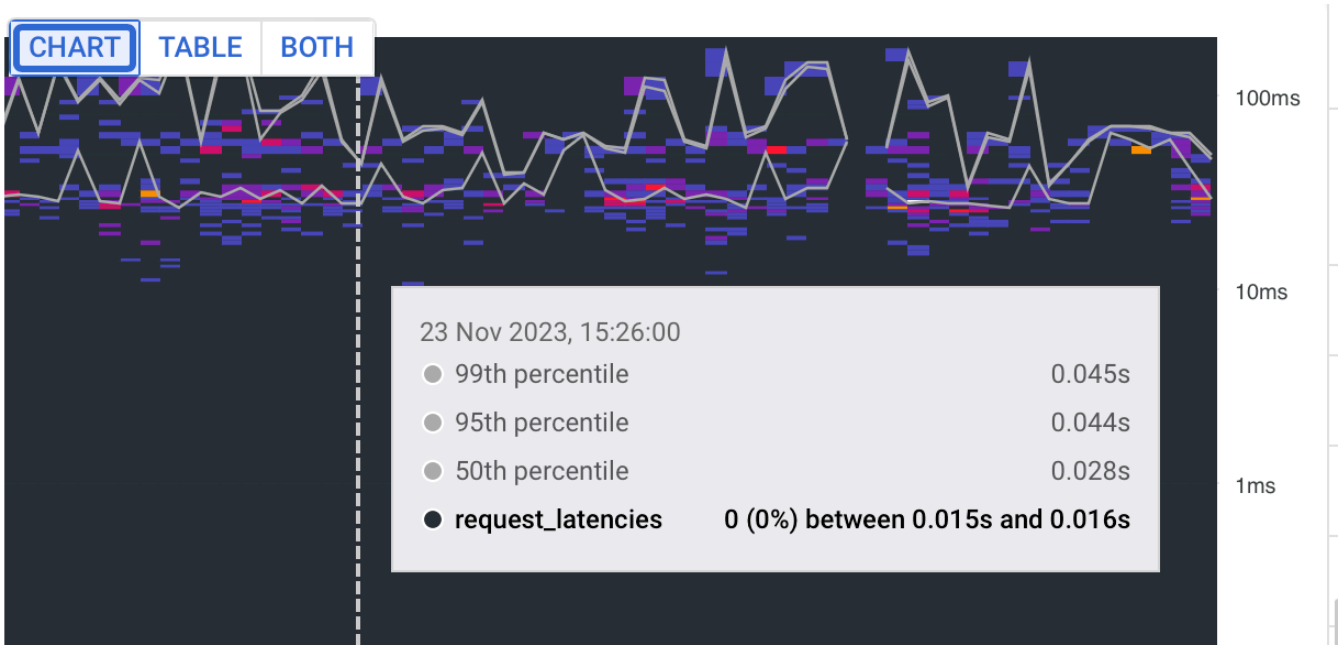
Measuring Latency Using Percentiles

Typically, latency is measured by percentiles, rather than averages, to provide a more accurate picture of the user experience. The 50th percentile (often referred to as the median) gives the latency experienced by the 'average' user. Higher percentiles, such as the 95th or 99th, provide insight into the latency experienced by the 'worst-off' users. These 'worst-case' latency numbers are often the focus when optimizing system performance. These are called “tail latency”.

- 50th percentile latency (aka median latency): The maximum latency (typically measured in seconds or milliseconds) for the fastest 50% of the requests.

- 99th percentile latency: The maximum latency (typically measured in seconds or milliseconds) for the fastest 99% of the requests.

Obviously, the 99th percentile latency will be greater than or equal to the 50th percentile latency. Here's an example from one of my projects using Firestore on Google Cloud.



Tail Latency

Tail latency refers to the high latency experienced in the worst-case scenarios, typically measured at the 99th, 99.9th, or even 99.99th percentile of a distribution.

In other words, if you were to measure the latency of thousands or millions of operations and plot them in ascending order, tail latency would represent the longest delays at the very end, or "tail," of this distribution.

In system design, tail latency is an essential metric because it often correlates with the worst user experiences. While average latency gives us a general idea of system performance, it doesn't account for occasional slowdowns that can have significant effects. This is why engineers focus not only on reducing average latency but also on minimizing tail latency.

Reducing tail latency (the high percentiles of latency) is a critical goal in system optimization, as it can significantly enhance the user experience. Tail latency refers to the slowest requests, which are often the most noticeable and disruptive to users. However, reducing tail latency can be quite challenging and often requires substantial effort and resources.

For instance, tackling tail latency might require optimizing server software, upgrading server hardware, or even altering system architecture. In some cases, it might involve improving load balancing or implementing better failover strategies to handle peak traffic or outages.

Balancing the effort to reduce tail latency with the benefits it brings is a critical aspect of system design. System designers need to consider the trade-offs involved and choose strategies that provide the best overall performance improvement for the resources available.

Managing Latency

Managing and reducing latency is crucial for maintaining an efficient and effective system. Here are some strategies system designers use:

1. Use of CDN (Content Delivery Network): CDNs store copies of data at multiple locations worldwide to reduce the distance data must travel, thereby reducing latency.
2. Load Balancing: By distributing network or application traffic across multiple servers, load balancers can help reduce congestion, improving response times and reducing latency.
3. Caching: By storing frequently accessed data closer to the user, caches can significantly reduce the time it takes to retrieve that data.
4. Choosing Appropriate Data Center Location: For cloud services, choosing a data center that's physically closer to the majority of users can help reduce latency. For example, if you are serving users in the US, you would want to pick your data center in San Francisco instead of Singapore.

Extra Readings

Google Cloud Spanner: Use metrics to diagnose latency